TP N°2 — Personnalisation et automatisation des VM avec Cloud-Init

Deux parties • 12 exercices • 2 séances de 3 h (durée indicative, prévoir du travail préparatoire)

Systèmes hôtes : Ubuntu 24.04 LTS ou Rocky Linux 9. Invités : images Ubuntu 24.04 ou Rocky Linux 9 compatibles Cloud-Init. Hyperviseur : KVM/libvirt du TP1.

Objectifs et prérequis

Réutiliser les images généralisées du TP1, injecter des configurations NoCloud, automatiser les utilisateurs, SSH, réseau et logiciels, puis générer des configurations multi-VM à partir de profils et de templates Jinja2.

Prérequis : KVM/libvirt opérationnels, réseau libvirt « default » actif, image QCOW2 arrêtée et généralisée, cloud-init installé dans l'image, accès à sudo et espace disque suffisant. Ne jamais modifier une image source pendant qu'une VM l'utilise.

Préparation commune (hôte)

Ubuntu 24.04 :

```
sudo apt update
sudo apt install -y cloud-image-utils genisoimage python3-venv python3-pip qemu-utils
virtinst libvirt-
clients
```

Rocky Linux 9 :

```
sudo dnf install -y genisoimage python3 python3-pip qemu-img virt-install libvirt-client
# Selon les dépôts, installer cloud-utils / cloud-utils-growpart si nécessaire
```

Outils à vérifier :

```
command -v genisoimage; command -v virt-install; command -v qemu-img
sudo virsh net-list --all
mkdir -p ~/bd2c/tp2/{nocloud,templates,profiles,inventory,generated}
```

Remarque : les images ISO NoCloud sont créées avec genisoimage, disponible dans les deux environnements. Les exemples ci-dessous utilisent une image de base dans /var/lib/libvirt/images/base-cloud.qcow2 ; adapter le chemin à l'image du TP1. Une image cloud officielle peut être utilisée si le template du TP1 n'est pas compatible.

PARTIE I — Initiation et configuration des VM (séance 1)

Exercice 1 — Découverte de Cloud-Init et de son architecture

Identifier les étapes init-local, init, config et final, les datasources et les modules Cloud-Init. Dans une VM compatible :

```
cloud-init --version
cloud-init status --long
cloud-init query ds 2>/dev/null || true
systemctl list-units "cloud-init*" --all
```

Questions — Quel est le rôle d'une datasource ? Quelle différence entre configuration au premier démarrage et gestion continue ?

Exercice 2 — Premier fichier user-data

Créer ~/bd2c/tp2/nocloud/user-data :

```
#cloud-config
hostname: bd2c-vm01
manage_etc_hosts: true
users:
  - default
  - name: etudiant
    groups: [wheel, sudo]
    shell: /bin/bash
    sudo: "ALL=(ALL) NOPASSWD:ALL"
    lock_passwd: true
```

Le groupe sudo existe sur Ubuntu, wheel sur Rocky ; la présence des deux dans l'exemple est à adapter à la distribution. Pour la connexion SSH, ajouter une clé publique à l'exercice 5.

Travail demandé — Expliquer les champs hostname, users, groups et lock_passwd ; vérifier la syntaxe YAML.

Exercice 3 — Datasource NoCloud et ISO seed

```
cat > ~/bd2c/tp2/nocloud/meta-data <<'EOF'
instance-id: iid-bd2c-vm01
local-hostname: bd2c-vm01
EOF
cd ~/bd2c/tp2/nocloud
genisoimage -output ../seed-vm01.iso -volid cidata -joliet -rock user-data meta-data
ls -lh ../seed-vm01.iso
```

Questions — Pourquoi instance-id doit-il changer lorsqu'on souhaite une nouvelle instance ? Quel est le rôle du volume « cidata » ?

Exercice 4 — Déployer une VM KVM personnalisée

Préparer un disque indépendant à partir de l'image de base arrêtée :

```
sudo cp /var/lib/libvirt/images/base-cloud.qcow2 /var/lib/libvirt/images/bd2c-vm01.qcow2
sudo virt-install --connect qemu:///system \
  --name bd2c-vm01 --memory 2048 --vcpus 2 \
  --disk path=/var/lib/libvirt/images/bd2c-vm01.qcow2,format=qcow2,bus=virtio \
  --disk path=$HOME/bd2c/tp2/seed-vm01.iso,device=cdrom \
  --network network=default,model=virtio \
  --import --os-variant generic --graphics none --noautoconsole
```

Attention : selon les permissions libvirt/SELinux, l'ISO dans $HOME peut être inaccessible à QEMU. Copier l'ISO dans /var/lib/libvirt/images avec sudo, puis utiliser ce chemin dans --disk. Une console série nécessite que l'image invitée soit configurée pour celle-ci ; sinon utiliser virt-manager.

```
sudo virsh list --all
sudo virsh domifaddr bd2c-vm01 --source lease
# Dans la VM :
hostnamectl; cloud-init status --wait
```

Travail demandé — Capturer le nom d'hôte et l'état Cloud-Init ; ne pas réutiliser un disque ayant déjà effectué son premier démarrage pour les tests suivants.

Exercice 5 — Utilisateurs et sécurisation SSH

Générer une paire de clés sur l'hôte, puis insérer le contenu de la clé publique dans user-data (sans jamais copier la clé privée).

```
ssh-keygen -t ed25519 -f ~/.ssh/bd2c_tp2 -C bd2c-tp2
cat ~/.ssh/bd2c_tp2.pub
```

```
users:
  - default
  - name: etudiant
    shell: /bin/bash
    groups: [wheel, sudo]
    sudo: "ALL=(ALL) NOPASSWD:ALL"
    lock_passwd: true
    ssh_authorized_keys:
      - ssh-ed25519 AAAA...REMPLACER_PAR_VOTRE_CLE_PUBLIQUE...
ssh_pwauth: false
```

Travail demandé — Reconstruire une nouvelle VM, tester ssh -i ~/.ssh/bd2c_tp2 etudiant@IP et expliquer pourquoi l'authentification par mot de passe est désactivée.

Exercice 6 — Configuration automatique du réseau

Créer network-config (format v2) ; relever au préalable le nom réel de l'interface dans la VM (ex. enp1s0).

```
version: 2
ethernets:
  enp1s0:
    dhcp4: true
```

Recréer l'ISO avec le troisième fichier :

```
genisoimage -output ../seed-reseau.iso -volid cidata -joliet -rock user-data meta-data
network-config
```

Travail demandé — Tester DHCP puis proposer une adresse statique adaptée au réseau libvirt, en vérifiant le plan d'adressage et en évitant les conflits. Sur Rocky, Cloud-Init peut appliquer la configuration via NetworkManager ; vérifier le résultat avec ip addr et ip route.

Exercice 7 — Paquets, fichiers et commandes

Compléter user-data avec les blocs suivants (fusionner dans un seul document YAML, sans dupliquer les clés) :

```
package_update: true
packages:
  - nginx
write_files:
  - path: /etc/motd
    content: |
      Laboratoire BD2C — Cloud-Init
    permissions: '0644'
runcmd:
  - [sh, -c, 'systemctl enable --now nginx']
```

Rocky Linux 9 : le paquet nginx doit être disponible dans les dépôts activés ; sur Ubuntu, il est installé avec apt. Selon la politique pare-feu de Rocky, ouvrir HTTP pour le test si nécessaire : sudo firewall-cmd --permanent --add-service=http && sudo firewall-cmd --reload.

Travail demandé — Tester systemctl status nginx, curl http://localhost et cat /etc/motd dans la VM.

PARTIE II — Industrialisation et déploiement multi-VM (séance 2)

Exercice 8 — Profils Cloud-Init réutilisables

Créer trois profils de services sous ~/bd2c/tp2/profiles : web.yml, db.yml, admin.yml. Exemple de profil :

```
# profiles/web.yml
packages:
  - nginx
service: nginx
role: web
```

Travail demandé — Définir les paquets et paramètres pour web, base de données et administration. Un profil est une donnée d'entrée ; il ne doit pas être confondu avec un fichier cloud-config complet.

Exercice 9 — Génération avec Jinja2

Installer Jinja2 dans un environnement virtuel Python sur l'hôte :

```
cd ~/bd2c/tp2
python3 -m venv .venv
source .venv/bin/activate
python -m pip install Jinja2 PyYAML
```

Créer templates/user-data.j2 :

```
#cloud-config
hostname: {{ hostname }}
manage_etc_hosts: true
users:
  - default
  - name: {{ username }}
    shell: /bin/bash
    groups: [wheel, sudo]
    sudo: "ALL=(ALL) NOPASSWD:ALL"
    lock_passwd: true
    ssh_authorized_keys:
      - {{ ssh_public_key }}
package_update: true
packages:
{% for pkg in packages %}
  - {{ pkg }}
{% endfor %}
```

Travail demandé — Identifier les variables et la boucle Jinja2 ; expliquer la séparation entre données et template.

Exercice 10 — Inventaire YAML et génération de plusieurs VM

Créer inventory/vms.yml :

```
vms:
  - name: bd2c-web
    hostname: web01
    username: etudiant
    profile: web
  - name: bd2c-db
    hostname: db01
    username: etudiant
    profile: db
  - name: bd2c-admin
    hostname: admin01
    username: etudiant
    profile: admin
```

Créer generate.py dans ~/bd2c/tp2 :

```
from pathlib import Path
import yaml
from jinja2 import Environment, FileSystemLoader
root = Path(__file__).resolve().parent
inventory = yaml.safe_load((root/'inventory/vms.yml').read_text())
env = Environment(loader=FileSystemLoader(root/'templates'), autoescape=False)
template = env.get_template('user-data.j2')
key = Path.home().joinpath('.ssh/bd2c_tp2.pub').read_text().strip()
for vm in inventory['vms']:
    profile = yaml.safe_load((root/f"profiles/{vm['profile']}.yml").read_text())
    target = root/'generated'/vm['name']
    target.mkdir(parents=True, exist_ok=True)
    (target/'user-data').write_text(template.render(**vm, **profile, ssh_public_key=key))
    (target/'meta-data').write_text(f"instance-id: iid-{vm['name']}\nlocal-hostname:
{vm['hostname']}\n")
    print(target)
```

```
python generate.py
find generated -type f | sort
```

Remarque : chaque profil doit comporter au minimum une liste packages. Pour les noms de paquets propres à Ubuntu ou Rocky, utiliser des profils distincts par distribution ou une variable os_family.

Travail demandé — Générer les trois répertoires, fabriquer une ISO NoCloud par VM, copier l'image de base trois fois, puis lancer trois instances distinctes avec virt-install --import. Adapter RAM et vCPU aux ressources disponibles.

Exercice 11 — Diagnostic et dépannage Cloud-Init

```
cloud-init status --long
cloud-init analyze show
sudo tail -n 80 /var/log/cloud-init.log
sudo tail -n 80 /var/log/cloud-init-output.log
sudo cloud-init schema --system
```

Tester un cas d'erreur volontaire (indentation YAML ou nom d'interface incorrect), corriger puis recréer une instance propre. La commande cloud-init clean peut réinitialiser l'état local dans une VM de laboratoire, mais elle ne remplace pas systématiquement un redéploiement depuis une image généralisée.

Questions — Pourquoi la même configuration ne se réexécute-t-elle pas toujours au redémarrage ? Comment distinguer une erreur de datasource d'une erreur de module ?

Exercice 12 — Mini-projet : infrastructure multi-VM personnalisée

Mission : créer trois VM depuis une même image de référence : web01 (Nginx), db01 (PostgreSQL ou MariaDB), admin01 (outils d'administration). Chaque VM reçoit un hostname, un utilisateur et une clé SSH ; son rôle est défini dans un profil. Les configurations sont générées automatiquement à partir de l'inventaire YAML et du template Jinja2.

Contraintes : noms et instance-id uniques, disques indépendants, absence de clés privées dans le dépôt, commandes reproductibles et vérification du réseau. Sur les ordinateurs limités à 8 Go, lancer les VM successivement et documenter les résultats.

Livrables attendus

1. Arborescence du projet ; 2. fichiers user-data, meta-data et network-config ; 3. trois profils YAML ; 4. template Jinja2 et script generate.py ; 5. inventaire des VM ; 6. commandes de création et de vérification ; 7. captures de cloud-init status, hostnamectl, ip addr, SSH et services ; 8. compte rendu avec réponses aux questions.

Barème proposé /20

Élément évalué	Points
Exercices 1 à 4 : fondamentaux et NoCloud	5
Exercices 5 à 7 : SSH, réseau, logiciels	4
Exercices 8 à 10 : profils, Jinja2, inventaire	5
Exercice 11 : diagnostic	2
Exercice 12 : mini-projet et reproductibilité	3
Qualité du compte rendu	1

Préparation du TP3 — Ansible

Conserver les adresses IP, clés SSH, profils et inventaires des VM. Au TP3, Cloud-Init réalisera la configuration initiale, tandis qu'Ansible assurera la configuration continue et le déploiement des services. Les inventaires pourront être adaptés au format Ansible.

Note enseignant : les durées sont indicatives. Préparer en amont une image cloud fonctionnelle pour chaque distribution et vérifier les contraintes de sécurité SELinux/AppArmor et d'accès aux disques libvirt sur les postes personnels.